

Diseño de Sistemas

Sistema: Conjunto de elementos relacionados (software y/o hardware) que trabajan para solucionar una necesidad.

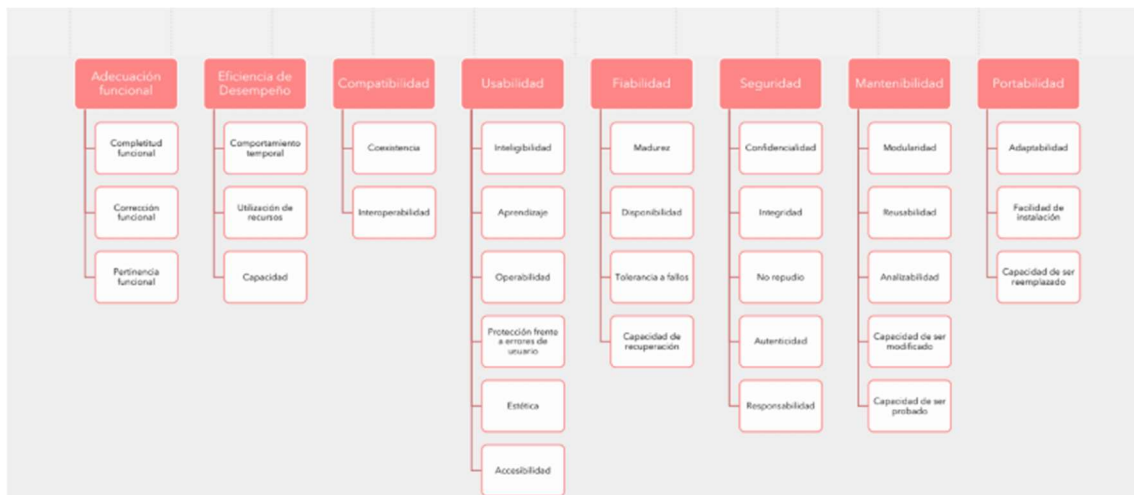
Diseño del sistema: Estrategia de alto nivel para la construcción de una solución. Se compone de módulos, interfaces, bases de datos y reglas de negocio, diseñados para cumplir con los requerimientos del usuario, tanto funcionales como no funcionales.

Requerimientos funcionales: Lo que el sistema debe hacer.

Requerimientos no funcionales: Como el sistema debe funcionar.

--

OBJETOS: CALIDAD DE SOFTWARE



PRINCIPAL DIFERENCIA:

- **MANTENIBILIDAD: MODULARIDAD – REUSABILIDAD – CAPACIDAD DE SER MODIFICADO**

CUALIDADES DE DISEÑO:

- **BAJO ACOPLAMIENTO: GRADO DE DEPENDENCIA ENTRE 2 COMPONENTES -> MANTENIBILIDAD | REUTILIZACIÓN**
- **ALTA COHESIÓN: + COHESIÓN – RESPONSABILIDADES**
- **ROBUSTEZ: ESTADO CONSISTENTE SIEMPRE, REPORTAR ERRORES, SE DEBE FACILITAR TANTO COMO SEA POSIBLE LA CAUSA DEL PROBLEMA**
- **FLEXIBILIDAD: EXTENSIBILIDAD (CAPACIDAD DE AGREGAR NUEVAS CARACTERÍSTICAS)**
MANTENIBILIDAD (CAPACIDAD DE MODIFICAR LAS CARACTERÍSTICAS EXISTENTES)



Persistencia y Modelo de Datos

Normalizar la información: La **normalización** es el proceso de estructurar una base de datos para reducir la **redundancia** y mejorar la **coherencia** de los datos, garantizando que las relaciones sean claras y mantengan su integridad. Se logra aplicando una serie de reglas y formas normales.

Desnormalización de la información: Consiste en **reintroducir redundancia** en una base de datos con el objetivo de mejorar el rendimiento de las consultas y reducir la cantidad de uniones (*joins*) necesarias para acceder a la información.

Bases de datos relacionales

Son aquellas que almacenan datos en tablas con filas y columnas, siguiendo el modelo relacional.

- **Modelo estructurado** basado en tablas con relaciones bien definidas.
- Uso de **claves primarias y foráneas** para mantener la integridad referencial.
- **Normalización** para reducir redundancia y evitar inconsistencias.
- **Transaccionalidad ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad).
- Operaciones de **JOIN** para combinar datos de múltiples tablas.

Desventajas:

- ✗ Puede **no escalar bien horizontalmente** debido a su estructura rígida.
- ✗ **Baja eficiencia en grandes volúmenes de datos** cuando se requieren múltiples joins.

Bases de datos NO relacionales

Son bases de datos que no utilizan tablas y están diseñadas para manejar datos semiestructurados o no estructurados con alta escalabilidad.

1. **Clave-Valor**: Datos almacenados como pares (clave, valor). Ej: Redis, DynamoDB
2. **Documentales**: Almacenan documentos en formato JSON/BSON. Ej: MongoDB, CouchDB
3. **Columnares**: Optimizadas para grandes volúmenes de datos distribuidos. Ej: Cassandra, HBase
4. **Grafos**: Diseñadas para modelar relaciones complejas entre datos. Ej: Neo4j, ArangoDB

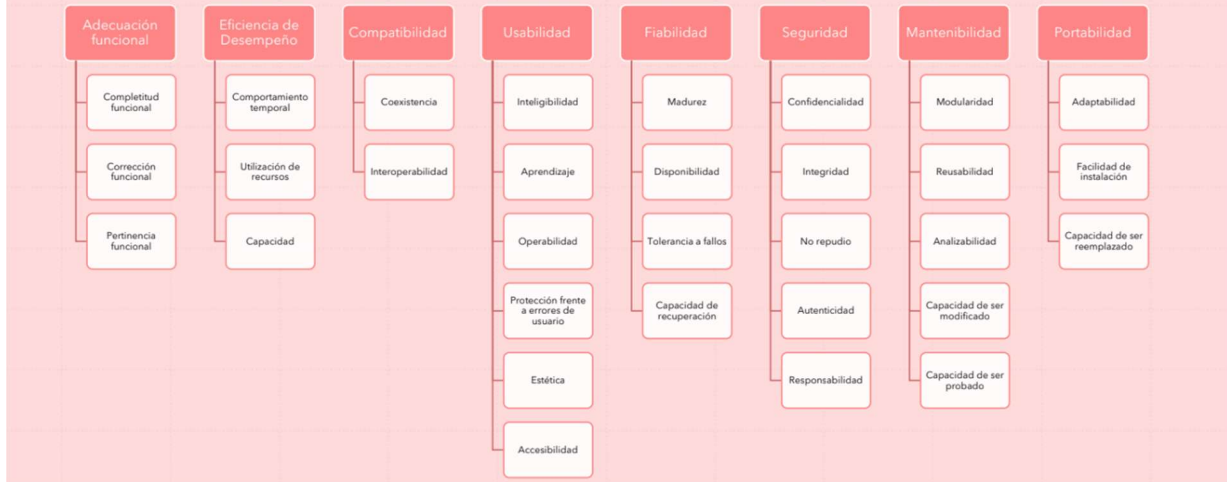
- **Mayor flexibilidad** en el almacenamiento de datos.
- **Escalabilidad horizontal** para manejar grandes volúmenes de información.
- **Alto rendimiento** en lectura y escritura.

Desventajas:

- ✗ No es ideal para **transacciones complejas** con múltiples relaciones.
- ✗ La **consistencia eventual** puede generar problemas en algunos casos.
- ✗ **Menos estandarización** en comparación con SQL.

Arquitectura

Atributos de Calidad (Arquitectónicos)



Ventajas

- Nos mantiene enfocados en el problema a resolver.
- Esconde el detalle de cómo se llevan a cabo los servicios expone.
- Puede ser reemplazada la implementación de los servicios siendo transparente para los consumidores.
- Minimiza la dependencia entre componentes.
- Facilita las pruebas.

Desventajas

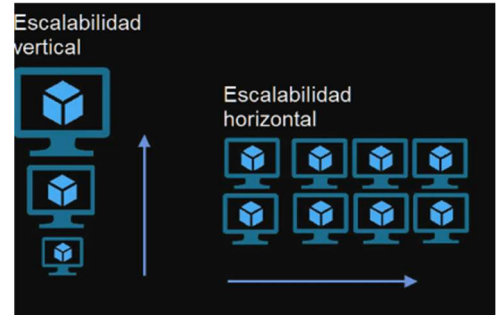
- Los cambios pueden generar efecto cascada.
- Demasiadas capas agregan complejidad y afectan negativamente al rendimiento.



que

Escalabilidad

Cuando un servidor debe atender muchas solicitudes, este se transforma en un cuello de botella y comienza a degradar la performance debido a que se registran tiempos de respuesta muy elevados. Incluso, hasta se puede afectar también la disponibilidad ya que, si dicho servidor se cae, todos los clientes quedarían sin poder realizar sus solicitudes.



Balanceador de carga

Aumenta complejidad, pero al escalar horizontalmente se debe crear este objeto nuevo que distribuya el tráfico entrante entre los servidores disponibles. Se comunican a través de IPs privadas, por lo que se suma una capa extra de seguridad. Pensar en estrategia de redundancia por si falla el balanceador ya que es un único punto posible de falla.

Sesiones

Una sesión es un mecanismo para mantener información y estado entre diferentes solicitudes y respuestas entre el cliente y el servidor. Durante la sesión, el servidor puede almacenar datos relevantes para el cliente, como información de inicio de sesión por ejemplo. Esto permite que el servidor recuerde al cliente y le brinde una experiencia personalizada.

Cookies

Las cookies son pequeños archivos que se almacenan del lado del cliente. Las cookies se envían automáticamente con cada solicitud al servidor, lo que permite que el servidor identifique al cliente y recupere información relacionada con la sesión. Las cookies son comúnmente utilizadas para mantener la autenticación del usuario, recordar preferencias, etc.

Arquitectura stateless vs arquitectura stateful

1. Evolución hacia arquitecturas escalables y distribuidas

- **Stateless** es la base de las **APIs RESTful**, microservicios y arquitecturas sin servidor (*serverless*).
- **Stateful**, aunque sigue siendo útil en algunos casos, presenta desafíos en entornos modernos donde la escalabilidad y la alta disponibilidad son clave.

2. Escalabilidad y Cloud Computing

- **Stateless** es más fácil de escalar horizontalmente (agregando más instancias de servidores) sin preocupaciones de sincronización de estado.
- **Stateful** requiere estrategias como replicación de sesiones o almacenamiento en caché distribuido (ej. Redis, Memcached) para escalar correctamente.

3. Integración con tecnologías modernas

- **Stateless** es el enfoque preferido en **API-first, microservicios, serverless y contenedores** (Docker, Kubernetes).
- **Stateful** todavía es común en aplicaciones monolíticas y sistemas tradicionales con gestión de sesiones en el servidor.

4. Ejemplos de uso

- **Stateless**: REST APIs, microservicios, AWS Lambda, Google Cloud Functions, aplicaciones de alto tráfico.
- **Stateful**: Aplicaciones con sesiones persistentes en memoria, videojuegos en línea con estado en el servidor, transacciones bancarias.

Consecuencias del uso de un balanceador de carga en una arquitectura stateful

La estrategia más simple sería utilizando el algoritmo de **sticky session**. Este algoritmo consiste en que las request de un cliente sean enviadas siempre al mismo servidor como se ve en la imagen.

Aunque, en el caso de que se caiga el servidor que tenía alojada la sesión para un determinado cliente el mismo experimente problemas en el uso del sitio. Por lo tanto, otra opción podría ser guardar toda la información que respecta la sesión en una BD (puede ser relacional o NoSQL) compartida entre todos los servidores; o bien en archivos compartidos.

API

Una interfaz de programación de aplicaciones (API) es un conjunto de herramientas, definiciones y protocolos que se usan para diseñar e integrar aplicaciones. Permite que un producto o servicio se comunique con otros productos y servicios, sin la necesidad de saber cómo se implementan internamente.

REST

Estilo de diseño de comunicación de Arquitectura a la hora de realizar una comunicación entre cliente y servidor.

- Crea una solicitud HTTP que contiene toda la información necesaria, es decir, un REQUEST a un servidor tiene toda la información necesaria y solo espera una RESPONSE, ósea una respuesta en concreto.
- Se apoya en los métodos básicos de HTTP, como son:

Post: Para crear recursos nuevos.

Get: Para obtener un listado o un recurso en concreto.

Put: Para modificar un recurso.

Patch: Para modificar un recurso (en forma parcial).

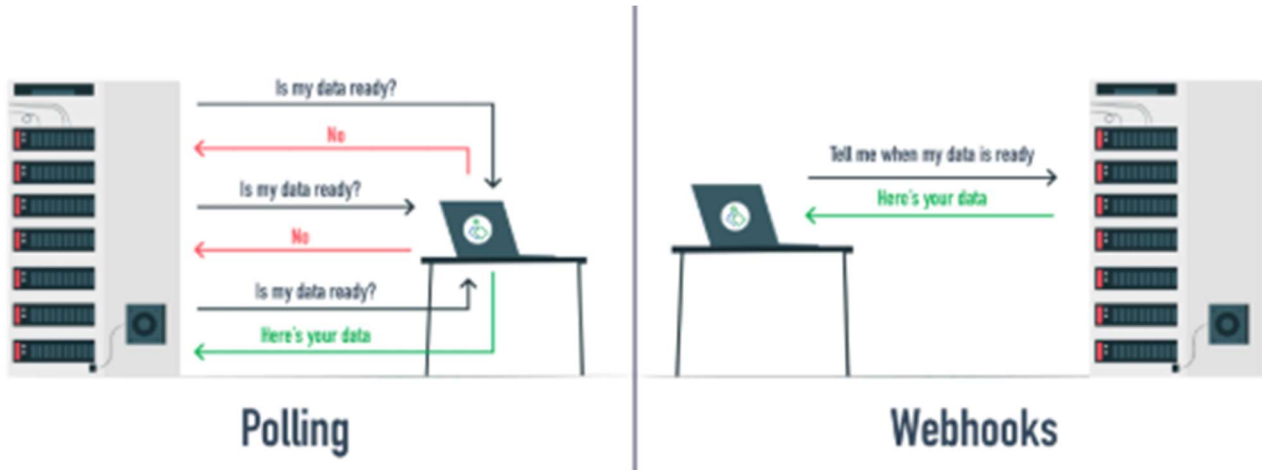
Delete: Para borrar un recurso.

Integraciones asincrónicas

Algunas de las formas que tiene A de enterarse, en un momento posterior, el resultado que generó el componente B.

Ante una mala utilización de este mecanismo podríamos generar una espera activa en A. -> A esta forma se la conoce como “pull based”

Webhook (push)

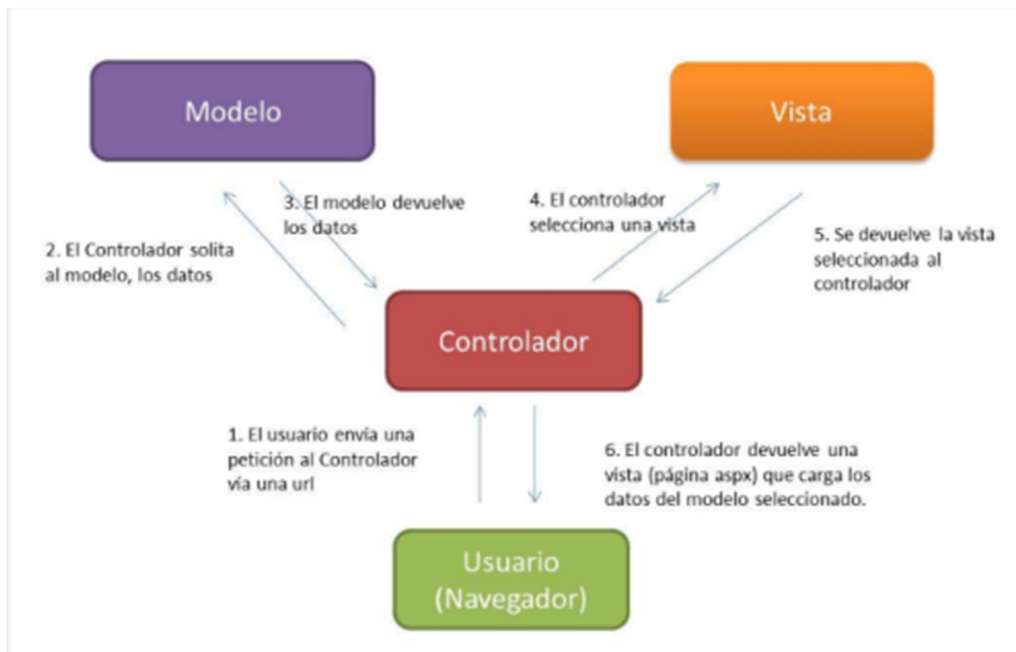


Un webhook es un endpoint de una API que actúa como un "escucha" para recibir notificaciones automáticas de eventos en otro sistema. Es decir, es una URL pública que un servidor externo puede llamar cuando ocurre un evento relevante.

Cronjobs

Un CronJob es una tarea programada que se ejecuta de manera periódica y automática. Se utiliza para automatizar comandos o scripts en intervalos de tiempo definidos por el usuario.

MVC



🚀 Atributos de Calidad en MVC

Atributo de Calidad	Descripción	Cómo lo logra MVC
Modularidad	Capacidad de dividir un sistema en módulos independientes.	Separando la lógica de negocio (Modelo), la presentación (Vista) y la gestión de peticiones (Controlador), lo que permite cambios en una capa sin afectar las demás.
Mantenibilidad	Facilidad para corregir errores o hacer modificaciones sin afectar el sistema.	La separación de responsabilidades permite que los cambios en una parte del sistema no impacten a otras, facilitando la depuración y evolución del software.
Reusabilidad	Posibilidad de reutilizar componentes en diferentes partes del sistema.	Un Modelo puede ser reutilizado por múltiples Vistas , por ejemplo, una misma API puede alimentar tanto una web como una app móvil.
Escalabilidad	Capacidad de un sistema para manejar un aumento en la carga de trabajo.	La separación de responsabilidades permite distribuir mejor la carga y mejorar la escalabilidad horizontal en aplicaciones web.
Separación de responsabilidades	Evitar la mezcla de lógica de negocio, presentación y control en un solo componente.	MVC define roles específicos: el Modelo gestiona datos, la Vista la interfaz y el Controlador coordina las interacciones.
Pruebas y Testabilidad	Capacidad de probar el sistema de manera modular.	Al separar la lógica de negocio del control y la presentación, se pueden realizar pruebas unitarias más efectivas en cada componente.
Flexibilidad	Facilidad para adaptar el sistema a cambios o nuevas tecnologías.	Se pueden cambiar vistas o controladores sin afectar el modelo de datos, permitiendo la adopción de nuevas tecnologías sin rediseñar toda la aplicación.

✗ **Mayor complejidad inicial:** Se necesita una curva de aprendizaje para entender la separación de capas.

✗ **Más archivos y estructura compleja:** Puede requerir más archivos y código que otros enfoques más simples.

✗ **No siempre es necesario:** Para aplicaciones pequeñas puede ser excesivo, ya que introduce sobrecarga en la organización del código.

Cliente liviano vs Cliente pesado

Liviano/Server side render	Pesado/Client side render
- Las vistas son generadas del lado del Servidor. - Las respuestas a las solicitudes son -casi- siempre HTML. - El navegador se encarga solamente de mostrar el contenido HTML. - En ciertos casos, se puede contener lógica escrita en JavaScript del lado del Cliente.	- Las vistas son generadas del lado del Cliente. - Las respuestas a las solicitudes son siempre JSON (o algún otro tipo de formato de intercambio similar). - El Cliente posee toda la lógica necesaria para presentar los datos recibidos, mediante código JavaScript.

	Cliente Liviano	Cliente Pesado
Generación de la Vista	Servidor	Cliente
Stream (dato que se transmite)	HTML	JSON
Procesamiento	- Mayor en Servidor - Menor en Cliente	- Mayor en Cliente - Menor en Servidor
Uso de Memoria	- Mayor en Servidor - Menor en Cliente	- Mayor en Cliente - Menor en Servidor
Lógica de Interacción	Servidor	Cliente
Usabilidad (*)	-	+
Escalabilidad (*)	-	+
Mantenibilidad (*)	-	+

Interactividad y Usabilidad	Permite experiencias más fluidas, sin recargas constantes de página (UX mejorado).
Escalabilidad	Reduce la carga del servidor, ya que el procesamiento ocurre en el cliente.
Eficiencia de Desempeño	Minimiza el tráfico entre cliente-servidor al enviar solo datos (JSON) en lugar de HTML completo.
Modularidad y Reusabilidad	Se pueden reutilizar componentes de frontend en diferentes partes de la aplicación.

Desventajas: Mayor carga en el cliente, Carga inicial más lenta

✦ Usa Cliente Pesado si:

- Necesitas alta interactividad y experiencia fluida (dashboards, apps tipo Gmail, Trello).
- Quieres reducir carga en el servidor y mejorar escalabilidad.
- Trabajas con APIs REST o GraphQL y manejas los datos en el frontend.

✦ Usa Cliente Liviano si:

- Tu aplicación es mayormente estática o basada en contenido (blogs, e-commerce tradicional).
- Quieres mejorar SEO sin necesidad de configuraciones avanzadas.
- Tienes recursos limitados en el servidor y prefieres centralizar la lógica.

Muchas aplicaciones modernas combinan ambos enfoques:

- SSR + SPA Híbrido: Frameworks como Next.js (React) y Nuxt.js (Vue) permiten renderizado en el servidor (SSR) + interactividad en el cliente.
- Backend con APIs y Frontend independiente: Se usa una arquitectura Headless, donde el backend solo expone APIs y el frontend consume datos.

SPA

Una **SPA (Single Page Application)** es un **tipo de Cliente Pesado** para manejar la navegación dentro de la misma página, sin recargar el navegador.

Experiencia fluida: No hay recargas completas de página, solo se actualizan partes específicas del contenido.

Mejor rendimiento en interacciones: Una vez cargada la aplicación, la navegación es instantánea.

Escalabilidad en la capa de datos

Escalabilidad horizontal vs Escalabilidad vertical

Escalabilidad vertical: Agregar recursos a la base de datos. Limitación física y es un único punto de fallo.

Escalabilidad horizontal: Complejo de implementar por mantener consistencia, se debe poseer un balanceador de carga para la distribución de las peticiones con algún mecanismo de actualización entre ellas.

Formas de escalar horizontalmente la base de datos

Replicación de base de datos

Mínimamente se posee 1 máster y una réplica. La base de datos es solo para escritura, y la o las replicas son solo de lectura. Es decisión del creador del sistema la lógica de cada cuanto tiempo se replique la información.

- **Rendimiento:** esta propuesta mejora el rendimiento ya que la cantidad de consultas que se procesan en simultáneo aumenta drásticamente debido a que las consultas se distribuyen en distintas bases de datos y no siempre las atiende la misma (máster)
- **Fiabilidad:** las bases de datos pueden encontrarse en distintos puntos geográficos, lo que garantiza que ante la pérdida de una por el motivo que fuese encontremos una copia de los datos en otra locación.
- **Alta disponibilidad:** El sistema puede seguir en funcionamiento si se muere un servidor con 1 de las bases de datos. Es decisión del creador del sistema decidir como actúa el mismo cuando esto pasa: Si falla una replica mandar la consulta al master y/o si falla el master transformar una replica en master...

Sharding o fragmentación

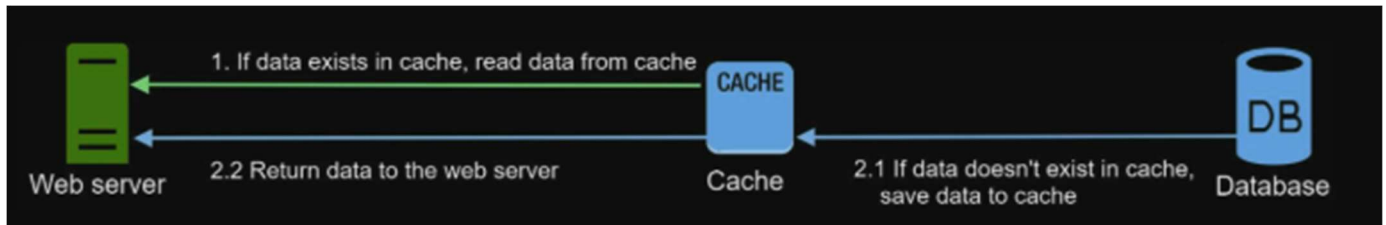
El proceso de sharding o fragmentación consiste en separar grandes bases de datos en muchas pequeñas que son más sencillas de manejar, donde a cada base de datos se la llama shard o fragmento. Todos los fragmentos tienen el mismo esquema pero no hay datos repetidos entre ninguno de ellos.

Posibles inconvenientes:

- Resharding de datos: por diversos motivos puede ocurrir que sea necesario hacer un resharding de los datos, es decir volver a organizar cómo se reparten los datos entre los distintos shards. Esto sucede cuando: un shard no puede contener más datos o ciertos fragmentos crecieron muy rápido debido a una distribución desigual de datos. Cuando esto ocurre se realiza un resharding actualizando la función de fragmentación. Es un proceso que puede llevar su tiempo y siempre que sea posible es mejor evitarlo.
- Problema de las celebridades: esto ocurre cuando datos que se utilizan frecuentemente son colocados en un mismo shard. Este shard tendrá tantos accesos que los usuarios experimentarían una respuesta lenta y hasta en ocasiones una caída del mismo. Es por eso que los datos que necesitan un acceso frecuente no deberían estar en el mismo shard. Incluso, podrían tener un shard exclusivo para alojar cada uno de esos datos en particular. Se lo conoce como problema de las celebridades ya que en el caso de una red social, las celebridades no pueden encontrarse en el mismo shard ya que esto provocaría una sobrecarga del mismo.
- Join y desnormalización: cuando los datos son fragmentados en distintas bases de datos es difícil realizar operaciones de unión. Una solución común a este problema suele ser la desnormalización de la base de datos para que las consultas se puedan realizar en una sola tabla.

Cache

La memoria caché es un almacenamiento temporal de datos cuya principal ventaja es la velocidad de acceso. Es por eso que se utiliza para almacenar datos a los que se acceden muy frecuentemente. La caché puede situarse en varios puntos de la arquitectura, como por ejemplo: entre el cliente y el servidor de aplicación, entre el servidor de aplicación y la base de datos, entre otros.



La caché de base de datos funciona de la siguiente manera: cuando el web server recibe una request se fija si la caché tiene almacenada la información suficiente para contestar la request, evita el acceso a la BD. En caso contrario envía un query a la base de datos y almacena el resultado en la caché.

- No es aconsejable almacenar datos en una caché de forma permanente, se recomienda la implementación de una política de caducidad. Es decir que los datos se borren cada un cierto tiempo, suficientemente largo como para que la cache cumpla su objetivo y lo suficientemente corto como para que los datos en la caché no queden desactualizados
- Se debe tener en cuenta que en el caso de que haya un fallo en la memoria caché, el sistema sepa cómo recuperar los datos de la fuente original ya que sino la memoria caché se transforma en un único punto de fallo (Single Point Of Failure)
- Por último, se debe también pensar en la política de desalojo. Una vez que la caché está llena se puede aplicar algún algoritmo como LRU (Last Recent Used), LFU (Least Frequently Used) o algo mas simple como un FIFO (First In First Out) para determinar cuál es el dato que se puede remover.

CDN (Content Delivery Network)

Una CDN es un tipo de caché. Consiste en una red de servidores dispersos por todo el mundo que se utilizan para proveer contenido estático como: imágenes, videos, css, JavaScript, etc

La CDN funciona proporcionando el contenido estático del servidor que se encuentre más cercano geográficamente. Por ejemplo: si hay un servidor en San Pablo y otro en Frankfurt la CDN se encargará de enviar el contenido desde el servidor de San Pablo a todos los usuarios que se encuentren en Buenos Aires, mejorando así la velocidad de carga del sitio.

Cola de Mensajes

Una cola de mensajes es un componente de software que admite la comunicación de forma asincrónica entre dos componentes de Software. La arquitectura básica de una cola de mensajes consiste en un productor/publicador que crea mensajes y los publica en la cola. Luego, otros servicios o servidores llamados consumidores/suscriptores se suscriben a la cola y comienzan a recibir mensajes de la misma que básicamente se trata de procesos a ejecutar. Es importante aclarar que los mensajes se procesan una sola vez por un único consumidor.



Las colas de mensajes se pueden usar para desacoplar procesos pesados y diseñar arquitecturas más escalables. Con la cola de mensajes, el productor puede enviar mensajes a la cola cuando el consumidor no está disponible para procesarlos (ya que el mensaje persiste en la cola hasta que un consumidor lo tome para procesarlo) y el consumidor puede leer mensajes cuando el productor está ocupado. Como consecuencia, esto significa que tanto el productor como el consumidor pueden escalar de forma independiente.

Logging y métricas

Logging: tener un registro de los logs del Sistema siempre es útil cuando queremos identificar un error o mejorar algún componente del sistema. Existen herramientas que nos proveen algunas funcionalidades interesantes para trabajar con los logs ya que en sistemas grandes suelen ser millones.

Métricas: las métricas son útiles para comprender la salud del sistema y nos da una idea si se están utilizando de forma eficiente los recursos disponibles o si esto puede ser el causante de un problema. Algunas métricas que son interesantes revisar son:

- Métricas a nivel de host: CPU, memoria, E/S de disco, etc.
- Métricas a nivel agregado: el rendimiento de la capa de la base de datos, la capa de caché, etc.
- Métricas clave del negocio: usuarios activos diarios, ingresos, etc.

Decisiones: **Aplicaciones Móviles: Nativas vs Híbridas**

Aplicaciones Nativas

Las aplicaciones nativas se desarrollan específicamente para un sistema operativo (Android o iOS) utilizando los lenguajes y herramientas oficiales de cada plataforma.

- **Android** → Kotlin, Java, Jetpack Compose.
- **iOS** → Swift, Objective-C, SwiftUI.

Aplicaciones Híbridas

Las aplicaciones **híbridas** utilizan una única base de código y se ejecutan en múltiples plataformas mediante tecnologías web (HTML, CSS, JavaScript) encapsuladas en un contenedor nativo.

- React Native (JavaScript, TypeScript).
- Flutter (Dart).
- Ionic (HTML, CSS, Angular/React).
- Apache Cordova (PhoneGap).

Característica	Nativa	Híbrida
Lenguaje	Kotlin, Swift	JavaScript, Dart, HTML, CSS
Rendimiento	Alto	Medio a alto (según optimización)
Experiencia de Usuario	Óptima	Puede tener diferencias con la UI nativa
Acceso a hardware	Completo	Limitado (depende del framework)
Tiempo de desarrollo	Mayor	Menor
Costo	Alto	Más económico
Mantenimiento	Doble código	Código único

Usa Nativo si:

- Necesitas **máximo rendimiento y optimización**.
- Tu aplicación requiere **uso intensivo de hardware** (cámara avanzada, AR/VR, sensores).
- Buscas la **mejor experiencia de usuario posible**.

Usa Híbrido si:

- Quieres **reducir costos** y mantener un solo código base.
- Tu app no depende de funcionalidades avanzadas de hardware.
- Necesitas **desarrollo rápido y actualizaciones frecuentes**.

Arquitectura Monolítica vs Arquitectura NO monolítica

Arquitectura Monolítica

Es un enfoque donde toda la aplicación es un **bloque único**, en el cual todos los componentes están integrados en un solo código base y comparten la misma base de datos.

Arquitectura No Monolítica

SOA (Arquitectura Orientada a Servicios)

En SOA, la aplicación se divide en **servicios más grandes y modulares** que interactúan entre sí mediante **protocolos estándar como SOAP o REST**.

Microservicios

Es un enfoque más moderno donde la aplicación se divide en **servicios pequeños, independientes y especializados**.

Componentes necesarios para una arquitectura No Monolítica:

Gateway API (Puerta de Entrada): Facilita la comunicación entre clientes (frontends) y los microservicios.

Balanceador de Carga (Load Balancer): Distribuye las solicitudes entre múltiples instancias de un servicio para evitar sobrecarga.

Service Discovery: Permite que los microservicios se encuentren entre sí sin necesidad de configuraciones fijas.

Comunicación entre Servicios:

- ✅ Sincrónica: API REST (HTTP), gRPC.
- ✅ Asíncrona: Mensajería con Kafka, RabbitMQ.

Bases de Datos Distribuidas: Cada microservicio maneja su propia base de datos para evitar dependencia entre servicios.

Gestión de Autenticación y Autorización: Garantiza la seguridad en el acceso a los microservicios.

SSO permite a los usuarios autenticarse una sola vez y acceder a múltiples sistemas sin necesidad de iniciar sesión repetidamente. (Single Sign On)

Monitoreo y Observabilidad: Permite detectar problemas en tiempo real en una arquitectura distribuida.

SOA vs Microservicios

Criterio	SOA	Microservicios
Tamaño del servicio	Servicios más grandes y monolíticos	Servicios pequeños y modulares
Independencia	Servicios acoplados a un ESB (Enterprise Service Bus)	Totalmente independientes, sin dependencia directa
Escalabilidad	Más difícil de escalar horizontalmente	Facilita la escalabilidad horizontal
Comunicación	Basada en SOAP, XML y mensajería en cola	APIs REST, gRPC, eventos con Kafka, RabbitMQ
Mantenimiento	Más difícil debido a la interdependencia de servicios	Fácil de mantener, cada servicio es autónomo
Despliegue	Despliegue más complejo, cambios afectan a múltiples servicios	Despliegue ágil, cada microservicio se actualiza de forma independiente
Base de datos	Base de datos centralizada y compartida	Cada microservicio maneja su propia base de datos
Tolerancia a fallos	Si el ESB falla, puede afectar toda la arquitectura	Mayor resiliencia, fallos en un servicio no afectan a los demás
Tecnología	Enfoque más rígido, usualmente Java, .NET, SOAP	Flexible, se pueden usar diferentes lenguajes y tecnologías en cada servicio
Curva de aprendizaje	Complejo de gestionar por la dependencia del ESB	Requiere conocimientos avanzados en escalabilidad y DevOps

SOA sigue siendo relevante para empresas que manejan sistemas legados, necesitan control centralizado o dependen de un ESB para integraciones complejas.

Microservicios son mejores para aplicaciones modernas, que requieren escalabilidad extrema y desarrollo ágil con equipos independientes.

Ejemplos de uso SOA:

Integración con Sistemas Legados (Sistemas Antiguos)

Muchas grandes empresas aún dependen de sistemas monolíticos heredados (Ej: ERP, CRM, Mainframes). SOA permite exponer funcionalidades antiguas como servicios reutilizables, sin necesidad de reescribir todo el sistema.

✗ Microservicios suelen requerir una refactorización completa, lo cual no siempre es viable en sistemas críticos.

🚀 Usa SOA si:

- ✓ Necesitas **integrar sistemas antiguos (ERP, CRM, Mainframes)** sin reescribir todo.
- ✓ Tu empresa ya usa **ESB** y **no tiene sentido migrar** a microservicios.
- ✓ **Se requiere compartir datos entre múltiples aplicaciones** de diferentes áreas.
- ✓ No tienes un equipo especializado en **DevOps, Kubernetes y CI/CD**.

🚀 Usa Microservicios si:

- ✓ Estás construyendo una **nueva aplicación escalable y moderna**.
- ✓ Quieres **desplegar y escalar módulos independientemente**.
- ✓ Puedes manejar la **complejidad de infraestructura** que implica.
- ✓ No necesitas una base de datos compartida globalmente entre todos los servicios.

🚀 Usa Monolito si:

- Es una aplicación pequeña o startup en fase inicial.
- No se espera un alto crecimiento en el corto plazo.
- Se busca simplicidad en el desarrollo y despliegue.

Atributos que pueden entrar en conflicto

Performance vs. Seguridad

- **Cómo entran en conflicto:**

Para mejorar la seguridad, se suelen agregar capas de autenticación, cifrado y validación de datos, lo que introduce latencia y afecta la velocidad de respuesta del sistema.

- **Ejemplo:**

Un sistema con autenticación multifactor (MFA) mejora la seguridad, pero agrega pasos adicionales que aumentan el tiempo de acceso. También, el cifrado de datos en tránsito y en reposo requiere procesamiento adicional, lo que puede afectar la rapidez con la que se accede a la información.

Seguridad vs. Disponibilidad

- **Cómo entran en conflicto:**

La seguridad muchas veces requiere controles estrictos que pueden limitar la disponibilidad. Por ejemplo, para proteger un sistema contra ataques DDoS, se pueden implementar medidas que restrinjan conexiones sospechosas, pero si la configuración es demasiado estricta, podría bloquear usuarios legítimos.

- **Ejemplo:**

Un firewall agresivo o un sistema de detección de intrusos (IDS) puede considerar tráfico legítimo como sospechoso y bloquear accesos, reduciendo la disponibilidad del sistema para algunos usuarios. Asimismo, los mecanismos de bloqueo tras múltiples intentos de login pueden dejar a usuarios legítimos sin acceso temporalmente.

Performance vs. Modificabilidad

- **Cómo entran en conflicto:**

Para lograr un alto rendimiento, se suelen hacer optimizaciones específicas, como la reducción de capas de abstracción o el uso de estructuras de datos altamente eficientes pero rígidas. Esto dificulta la modificabilidad, ya que cualquier cambio puede ser más costoso o afectar el rendimiento.

- **Ejemplo:**

Un sistema con código muy optimizado en C++ para mejorar la velocidad puede ser difícil de modificar o extender. Si se usara un diseño más modular con patrones de diseño orientados a la extensibilidad, el sistema sería más fácil de modificar, pero posiblemente menos eficiente en términos de rendimiento.

Diagrama de despliegue

Concepto	Definición	Ejemplo
Nodo	Representa una entidad física o virtual donde se ejecutan componentes de software.	Un servidor, una máquina virtual, un contenedor Docker.
Componente de Software	Representa una pieza de código ejecutable dentro de un nodo. Puede ser un microservicio, una API, una base de datos.	Un servidor web Nginx, una API en Node.js, una base de datos PostgreSQL.

Protocolos entre componentes

Protocolo	Uso en Arquitectura	Ejemplo de Aplicación
HTTP/HTTPS	Comunicación entre clientes y APIs REST.	Un navegador accediendo a una API REST.
TCP/IP	Conexión entre servidores, bases de datos y microservicios.	Comunicación entre un backend en Java una base de datos PostgreSQL.
AMQP (Advanced Message Queuing Protocol)	Comunicación en colas de mensajería.	RabbitMQ, procesamiento de eventos asíncronos.
MQTT (Message Queuing Telemetry Transport)	Comunicación en sistemas IoT.	Sensores que envían datos a la nube.